

> Exercícios CAP. 1 (pág. 75) <

- ① Ser um intermediário que facilita a interação entre o usuário e o hardware, sendo um gestor de todos os recursos da máquina.
- ③ Multiprogramação foca na eficiência de hardware, garantindo que a CPU não fique ociosa. Já o compartilhamento de tempo faz com que a CPU alterne o contexto entre as tarefas de forma tão rápida que cria a ilusão de pseudoparalelismo.
- ⑤ Sem o DMA, a CPU é forçada a gerir manualmente cada byte transferido para os dispositivos de E/S. Logo, se a CPU precisar parar, ela não pode executar o código de outros processos ao mesmo tempo, destruindo o ganho de eficiência e o pseudoparalelismo proposto pela multiprogramação.
- ⑥ Restringindo, a aplicação precisa realizar uma chamada de sistema para transitar de modo utilizador para núcleo, garantindo que o SO valide a permissão e o destino da operação antes de executar.
- ⑨ Um dilema é o conflito entre Vazão e Tempo de Resposta. Se o escalonador utilizar um quanta muito curto no algoritmo Round-Robin, o sistema responderá de forma instantânea, mas gastará muito poder computacional para trocas de contexto, diminuindo a vazão total de processos finalizados.
- ⑩ O modo usuário é um ambiente restrito onde o código executa sem acesso direto ao hardware ou áreas críticas da memória. Já o modo núcleo, possui privilégio total de instrução e endereçamento. Esses dois modos ajudam a isolar falhas, impedindo que o sistema inteiro colapse (por exemplo: divisão por zero).

(14) Ciclo de clock = 1 ns
1 instrução $\Rightarrow$ 1 ns

(19) Sim, pois ao fazer isso os arquivos originais ficam temporariamente escondidos / protegidos, sendo possível assim testes sem apagar a raiz.

(28) Porque as chamadas de sistema custam tempo de CPU porque exigem a mudança de contexto entre usuário e núcleo. Um código que faz mil chamadas de sistema será bem mais lento que um código feito para chamadas pontuais.

(29) Para portabilizar aplicações, o programador precisa ter o cuidado de redesenhar as funções, implementando uma lógica extra para simular o estado, o que complica o controle e a sincronização do código.
